

Evidence Crosses, Authority Does Not: Receiver-Owned Admission of Agent Tool Calls Within and Across Organizations

The Chio Project

<https://github.com/bb-connor/chio>

September 2026

Abstract

An agent calling a tool through an ordinary client holds whatever authority its host process holds; across an organization boundary the receiver has only the sender's word. Our protocol rests on one rule: authority is minted and verified only inside the kernel that dispatches, against roots it activated itself, and nothing in a request can add a root; only signed, content-addressed evidence crosses. What crosses is the receipt, a kernel-signed record of one admission decision named by its content hash. Two co-signed statements name receipts. The inbound one is evidence a request presents about a receipt the receiver already holds; resolving it before dispatch is where the security property lives. The outbound one the receiver mints over its own new receipt once that receipt is durable, and the peer signs those bytes unparsed; it gates the response, never the dispatch. Neither is a bearer credential: a statement names only what the receiver stores. Which names must cross is settled by what the receiver decides, so the carrier is co-designed with the decision rather than bolted onto an existing request. We enumerate them, mechanize the classification, and measure what a composition of a pinned peer identity, a signing policy engine, and a replay table must add to reach each property.

1 The rule

A buyer agent at one company asks a vendor agent at a second to stage a refund against the vendor's payment ledger. The vendor's software must decide, before touching it, whether to obey. An API key and a transport signature establish who is calling and nothing about whether this call, with these arguments, under the agreement the two companies signed, was already authorized, already performed, or rests on a delegation revoked minutes ago. Afterward each holds a log about itself that no third party can reconcile. Having the sender prove more makes the receiver's security a function of the sender's correctness; a shared ledger or broker makes that substrate a participant in every call.

The rule here avoids both. **Authority is minted and verified only inside the kernel that dispatches, against roots that kernel activated itself, and nothing carried in a request can add a root. Only signed, content-addressed evidence crosses an organization boundary.** A signature proves who signed some bytes; it cannot decide whether the receiver should act on them, because that depends on facts only the receiver holds: which keys it pinned, which agreement it activated and when, whether this continuation has been spent.

That is a placement argument, composing with the stack, not replacing it [16]. Alignment training, transcript monitors, and the sender's policy engine change how often the kernel says no; none knows the callee's budget is exhausted or this argument hash already dispatched. The receiver decides.

One object carries it: the receipt, a kernel-signed record of one admission decision named by its own content hash. Two co-signed statements name receipts. The **inbound** statement is evidence a request presents about a receipt the receiver already holds, signed by two keys it pinned beforehand; the pre-dispatch hook resolves it against the receiver's own stores, where the security property lives. The **outbound** statement the receiver mints over its own new receipt, once that receipt is durable, and the peer signs those bytes unparsed; it gates the response, never the dispatch. Neither is a bearer credential: what either names the receiver stores. Agent protocols define how a tool is called [2, 1]; none defines the record of what was admitted. We call the mediating component the kernel: Chio is userspace, not an operating-system kernel.

Which names must cross is settled by what the receiver decides. Each fact one decision binds needs a field to arrive in, so the carrier is co-designed with the decision rather than bolted onto a request shaped before it. Section 8 measures that against a composition of a pinned peer identity, a signing policy engine, and a replay table: what an operator must add to reach each property of Section 6, in fields, checks, and order.

2 Threat model and trusted base

The kernel sits between an agent and the tool servers whose transport it owns: child processes, fronted HTTP tools, proxied sidecars. It does not confine the agent, whose model, orchestrator, prompt, and libraries are untrusted; one that opens its own socket reaches whatever that socket reaches.

The adversary is given everything outside the receiving kernel: every field this paper names, the peer kernel and its signing key, the network, where it may drop, reorder, and replay, and any receipt, statement, lease, or governance record it has observed. It holds neither the receiving kernel’s signing key nor write access to its stores. The question is what it can cause a receiver to dispatch.

The trusted base is the receiving kernel’s process, its signing key, and its stores; everything outside is verified or assumed; the assumptions appear once, in Table 1.

Table 1: Assumptions outside the receiving kernel, by the identifiers of the implementation’s assumption registry. The registry has 16 entries; the ten this protocol rests on are listed.

Identifier	What is assumed
ASSUME-ED25519	Ed25519 meets existential unforgeability under chosen-message attack for keys the kernel trusts.
ASSUME-SHA256	SHA-256 is collision resistant over receipt and statement preimages.
ASSUME-CANONICAL-JSON	The production canonical encoder agrees byte for byte with the mechanized one on strings, bounded integers, finite arrays, and objects ordered by UTF-16 code unit. Floats are outside that domain.
ASSUME-SQLITE-ATOMICITY	A single-row insert commits atomically, so a primary-key conflict is a reliable witness that a row existed. Cross-row recovery and ordering are not assumed.
ASSUME-OS-CLOCK	The injected clock reports Unix time within the operator’s declared tolerance.
ASSUME-NETWORK-TRANSPORT, ASSUME-TLS	Delivery is not assumed reliable; arriving messages are assumed not rewritten below the signature checks, and remote surfaces authenticate their endpoints.
ASSUME-SUBPROCESS-ISOLATION	Effects after an allow, and isolation between tool processes, rest on operating-system boundaries.
ASSUME-FEDERATED-ORIGIN-CLASSIFICATION	The receiving edge marks cross-organization origin from a transport-authenticated peer identity; a request misclassified as local never reaches these checks.
ASSUME-GOSSIP-FAIRNESS-PARTITION-BOUND	Revocation liveness only: recurring delivery under weak fairness, bounded skew, operator-bounded partition healing. Safety does not depend on it.

Two assumptions are deliberately absent. The first is that the two signing keys on a co-signed statement belong to different organizations: one operator holding both satisfies every check here. The second is domain separation: one passport key signs three preimage families, the receipt signing body, the co-signing body, and a statement’s pre-authentication encoding, so ASSUME-ED25519 is an assumption about a key with that reuse. A pinned peer obtains this kernel’s signature over any bytes that rebuild as one of the last two, so a countersignature fixes a record’s shape and parties, not its truth.

3 The receipt

A receipt is a JSON object with a fixed field set, encoded canonically under RFC 8785 [15]. Three of its projections are hashed, and they nest: b drops the `id`, the `signature`, and the two optional fields recording how it was signed; the canonical receipt body β is b plus the `id`; r is the whole receipt as it travels. Write D for SHA-256 of the canonical encoding and k for the kernel key published in the body; signatures are Ed25519 [12].

$$id = D(b), \quad \sigma = \text{Ed25519}_k(\text{JCS}(\{\text{body}: b, \text{id}: id\}))$$

Each projection has its own wire field: id names the receipt, $D(\beta)$ is the subject digest of a co-signed statement, inbound or outbound, and $D(r)$ is a binding reference’s `remote_receipt_sha256`. No two agree, because β adds the `id` to b and r adds σ to β . The signature covers a fourth preimage, the object above, which the optional BBS signature joins as a third key, so that field is covered by σ and not by id . A receipt renamed is a receipt unsigned. Three properties follow.

Recompute and refuse. Where the path holds the evaluated content the signer is handed the content, not a digest: it recomputes the digest inside the trust boundary and refuses to sign if the body’s claimed digest differs.

Decisions, not successes. A denial is a receipt of the same shape, built, signed, and persisted on the same path as an allow. Three fields fix what kind of statement a receipt is, so a weaker one cannot pose as a

stronger one: `receipt_kind` separates a mediated decision from a passive trace, `boundary_class` says whether the kernel could have prevented the effect or only observed it, and `trust_level` records the evidence class. A body claiming a decision on a kind that cannot make one is not signed; a message rejected before it had a decision to record returns its code and no receipt.

Permission, proof, and price in one object. When the exercised grant carries monetary caps the hold taken before dispatch is reconciled into the receipt’s financial block: `cost_charged`, `currency`, `budget_remaining`, `budget_total`, the delegation depth, and the root budget holder. The receipt an auditor reads to learn the call was allowed is the object a biller reads to learn what it cost. Settlement between organizations is out of scope, and that is our whole claim about commerce.

The vendor kernel’s receipt for the refund step, abridged at the metadata:

```
{ "id": "d342a59d...6fba2e3f", "timestamp": 1766000001,
  "capability_id": "lease-vendor-c-refund", "tool_server": "vendor-c.payments",
  "tool_name": "stage_refund",
  "action": { "parameters": { "caseRef": "refund-250", "tool": "stage_refund",
    "workflowId": "wf-chio-refund-001" },
    "parameter_hash": "47e6e096...d5a6464b" },
  "decision": { "verdict": "allow" }, "trust_level": "mediated",
  "receipt_kind": "mediated_decision", "boundary_class": "prevent",
  "tool_origin": "caller_executed", "redaction_mode": "none",
  "content_hash": "c9abf68a...d9803987",
  "policy_hash": "82cfeae3...fb2ca3fd",
  "metadata": { "attribution": { "delegation_depth": 0, "grant_index": 0, ... },
    "chio_runtime": { "accepted": true, "destructive": true,
      "lease_id": "lease-vendor-c-refund", "step_index": 2,
      "reserved_treaty_continuation_id": "continuation:runtime-loopback:2" }, ... },
  "kernel_key": "31debe55...cfa498bc",
  "signature": "a9e4762e...1ba28505" }
```

Checked on these bytes: b hashes to `d342a59d...6fba2e3f`, the object’s id; β to `31118bb6...4781b3f9`, the subject digest of the statement the vendor mints over it after persisting it (Section 5); r to `63f1f35a...8c93e3e8`, that statement’s `remote_receipt_sha256`. The signing preimage over b verifies under the embedded kernel key. This receipt carries no financial block because the exercised grant carries no cap.

4 Admission

Admission is the one operation. Every tool call, local or foreign, goes through it, and every path that reached a decision produces a signed receipt.

1. Verify the presented capability against the kernel’s own trusted issuer set: signature, time window, subject binding, and the revocation state of the leaf and every ancestor; resolve the grants matching this tool and these arguments.
2. Open a durable admission operation, a fenced write-ahead record whose state machine has 18 states. Its fence is a store lease with an epoch: no second coordinator can advance it.
3. Run the guard pipeline. Deny dominates, and so does error: a guard that produces no verdict denies.
4. Run the admission hook, where Section 5 runs for a request classified as foreign; one carrying no agreement context is denied here.
5. Take the budget hold, charging the reservation before dispatch.
6. Revalidate the mutable inputs, mark the operation dispatch-committed, and only then hand the call to the tool server over a transport the kernel owns.
7. Run output guards, build the receipt, recheck its fields against the admitted inputs, sign and persist it, then return.

Three orderings are the whole of its value. **Durable before dispatch:** no dispatch occurs without a committed pre-dispatch record. Before that commit recovery releases the hold; after it, with no outcome known, recovery terminates the operation as outcome-unknown rather than inventing one. An output not matching the committed digest terminates as denied-after-delivery, with a signed deny and nothing captured against the hold.

Receipt before response: the receipt is persisted before the caller sees the result, and a kernel that cannot persist a receipt denies. The log is a write-ahead record of what was admitted, not a trace of what was observed.

Consume after compare: step 4 spends a foreign request’s single-use continuation after every comparison against the inbound statement it presented, and after both signatures on that statement verify, and before the local admission report that rebuilds this call’s binding from the live request. A statement the receiver refuses cannot burn a continuation a legitimate call still needs. A statement that passes every comparison on

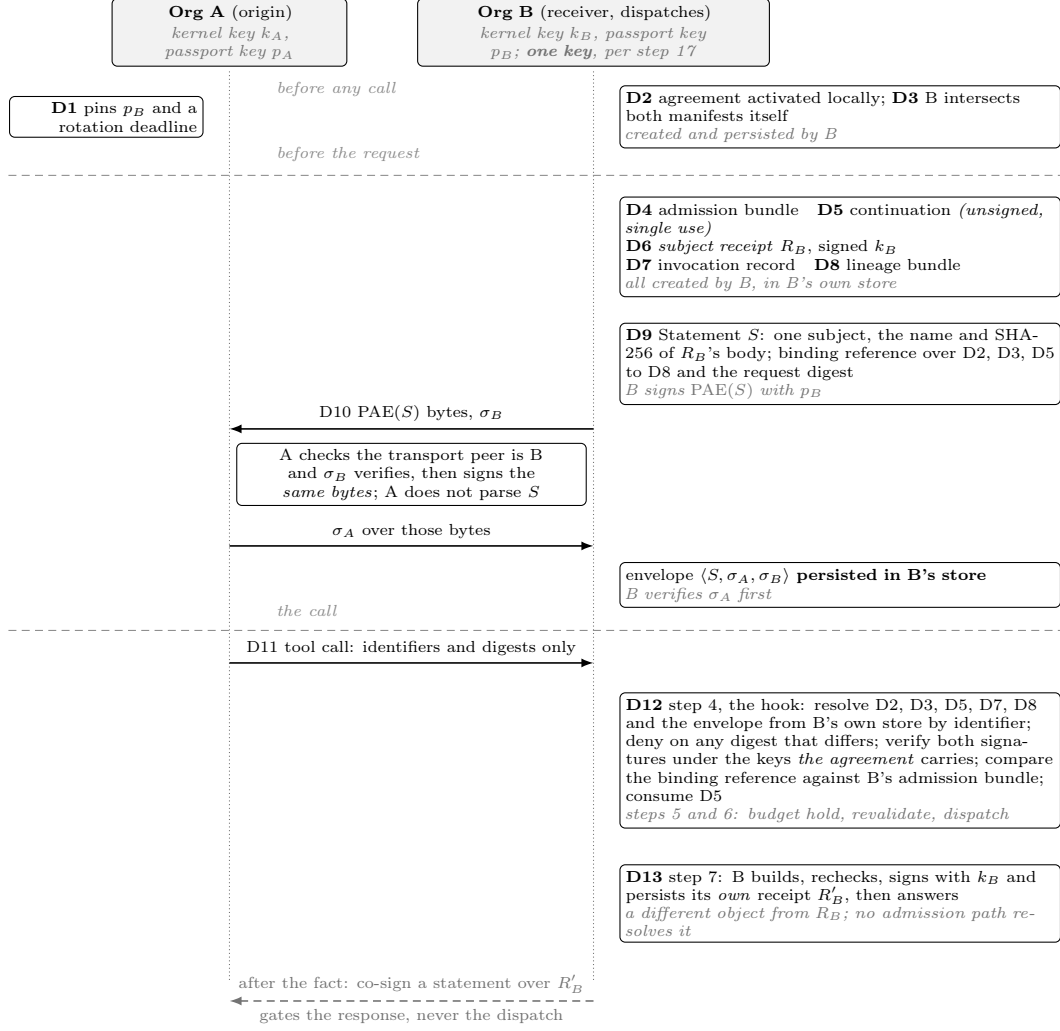


Figure 1: One cross-organization call, end to end. Every object the receiving kernel checks at step 4 was created by the receiver and was already in the receiver's own store before the request arrived; the request carries identifiers and digests and nothing else. The statement the hook verifies (D9, D10) has as its subject a receipt the *receiver* signed (D6), not the peer's, and that is a different receipt from the one the receiver writes for this call at step 7 (D13). The dashed return is the outbound co-signature: the reference kernel mints it on every federated request that produced verified treaty material and fails the response if it cannot, but it is minted after the dispatch it describes and no admission path resolves it.

a request the report then denies does spend one, and gets it back through a release scoped to that admission; an unconfirmed release is recorded as ambiguous rather than released.

5 Cross-organization admission

A call from another organization is the same admission operation, and both co-signed statements occur in it. The inbound one is named by the request and resolved by the pre-dispatch hook at step 4; it adds no authority, because the receiver's issuer set, pinned peers, agreements, and clocks are what they were before the request arrived. Section 6 concerns it; the outbound one closes this section, and Figure 1 follows one call end to end.

5.1 Receiver-owned resolution

Two organizations pin each other out of band, each recording the other's kernel identity, the passport key it co-signs with, and a rotation deadline. The agreement is an activated record naming its participant kernels,

the action classes in scope, a validity window, and a revocation epoch; the specification says exactly two, the validator accepts more, and exactly two is enforced where co-signed evidence is checked. Each participant publishes a signed manifest, the action classes it accepts and at what intensity on the ascending ladder observation, guarded, receipt_backed, partition_contingency, maintenance, and each receiver activates the agreement and both manifests locally and intersects them per action class itself: the higher-intensity mode wins, the stricter co-signing requirement wins, and a destructive class below receipt_backed rejects. A pin the receiver has not activated is a denial.

The request may carry identifiers. It may not carry trust. Its agreement context is names and digests, each a lookup key into a store the receiver owns; Figure 2 is the whole of what such a name reaches. Eleven further key names are refused as keys of that context, seven naming a trust root, bundle, agreement record, manifest, signing key or peer directory and four naming dynamic trust input or peer discovery, with request_smuggled_trust_root and request_smuggled_dynamic_trust; nested deeper they are unread, not refused.

5.2 The co-signed statement

One wire form serves both directions: an in-toto Statement v1 [21] in a DSSE envelope [18] whose predicate is snake_case and rejects unknown fields. Its single subject names an invocation identifier and carries the SHA-256 of that receipt's canonical body of Section 3. Below is the vendor's outbound refund envelope and its payload. An inbound subject instead names a receipt the receiver already held, in the reference path one record per process on every call, so the per-call binding rests on the binding reference alone.

```
{ "payloadType": "application/vnd.in-toto+json",
  "payload": "eyJfdHlwZSI6Imh0dHBzOiI8vaW4tdG90by5pby9T...",
  "signatures": [
    { "keyid": "fdf72a088f18f7399e8c52bce448441501f759a595a86980e5d9a422a01e5d55",
      "sig": "3QZ09JP4YLFmvgzwy5PwqYTFCCQkUo/DRc7YyFB8pD+bA6iBGwyRTtI+1zzI7A..." },
    { "keyid": "e37abcd79efd08db94518a903952ffe9f8af7e8b4684b63591631cef8e8871c8",
      "sig": "R0Ii8tcBzYcEPAPRQqNmbCUowFH0hzpjr5Mm98mFu33+XcT0fDxdWf48btYzmd..." } ] }
```

```
{ "type": "https://in-toto.io/Statement/v1",
  "predicateType": "chio.bilateral-cosign-invocation.v1",
  "subject": [ { "name": "chio-receipt:d342a59d...6fba2e3f",
    "digest": { "sha256": "31118bb6...4781b3f9" } } ],
  "predicate": {
    "invocation_id": "d342a59d...6fba2e3f", "tool_name": "stage_refund",
    "tool_server_a": { "kernel_id": "did:chio:buyer-kernel", "alg": "ed25519",
      "passport_key_fingerprint": "fdf72a08...a01e5d55" },
    "tool_server_b": { "kernel_id": "did:chio:vendor-c", "alg": "ed25519",
      "passport_key_fingerprint": "e37abcd7...8e8871c8" },
    "co_sign": "bilateral_required", "consistency_model": "totally-ordered",
    "consistency_anchor": "chio:consistency:wf-chio-refund-001:2",
    "cross_org_visibility": "federated", "timestamp_unix_ms": 176600001000,
    "tool_args_hash": { "alg": "sha256", "value": "47e6e096...d5a6464b" },
    "capability_lease_ref": { "lease_id": "lease-vendor-c-refund",
      "issuer": "did:chio:buyer-kernel", "expires_at_unix_ms": 1766000030000,
      "scope_digest": { "alg": "sha256", "value": "262768fe...b9f835aa" } },
    "governance_receipt_ref": { "receipt_id": "gov-refund-stage-authorization",
      "kernel_id": "did:chio:buyer-governance",
      "digest": { "alg": "sha256", "value": "21143a93...f161e3c0" } },
    "policy_evaluation_summary": { "joint_disposition": "allow",
      "server_a_verdict": { "verdict": "allow", "policy_id": "buyer-policy:stage_refund", ... },
      "server_b_verdict": { "verdict": "allow", ... } },
    "treaty_binding_ref": {
      "treaty_id": "treaty:runtime-loopback:2",
      "action_class_id": "workflow.cross_kernel.stage_refund",
      "consistency_model": "totally-ordered",
      "treaty_scope_sha256": "0ff230e9...dif2b21c",
      "ladder_intersection_sha256": "93dcc7cf...af79d3d1",
      "admission_report_sha256": "91d74509...1f646522",
      "continuation_sha256": "8ef2f348...366b93f4",
      "lineage_bundle_sha256": "28d816a3...d3b504ba",
      "request_sha256": "47e6e096...d5a6464b",
      "outcome_sha256": "c9abf68a...d9803987",
      "local_receipt_sha256": "9cbe4438...ad017121",
      "remote_receipt_sha256": "63f1f35a...8c93e3e8",
      "lease_refs": [ "lease-vendor-c-refund" ],
      "governance_refs": [ "gov-refund-stage-authorization" ],
      "signer_kernel_ids": [ "did:chio:buyer-kernel", "did:chio:vendor-c" ] } }
```

Both signatures cover the DSSE pre-authentication encoding of the payload, and each keyid is the SHA-256 of its raw public key. Recomputed from the committed bytes, request_sha256 is tool_args_hash.value and the receipt's action.parameter_hash.

5.3 The verifier, in the order it runs

Table 2 is the shipped order; two components decide, in two disjoint code families. The conforming verifier returns the specification's dotted codes, a closed set of 24 strings of which it emits 16 and the envelope layer the rest. The kernel's pre-dispatch hook answers from its runtime failure-code set of 505 strings, 24 of

Table 2: The conforming verifier, 26 steps, grouped by where the state each step reads comes from. Every failure returns one code from the closed set; the specification maps each step to its own. Steps 8 to 14 are the envelope layer, called with the two keys the receiver pinned.

#	Check
1–5	Media type, at least one signature, payload decodes and parses; predicate type chio.bilateral-cosign-invocation.v1 ; Statement v1; exactly one subject
6–7	Both kernel identifiers are in the pinned peer set and each pin’s fingerprint equals the declared one
8–13	Exactly two signatures; re-canonicalizing reproduces the payload bytes; predicate schema and strict rules; both verdicts present, equal, and in {allow, deny}; subject name is chio-receipt: plus the invocation identifier; the two pinned keys and their identifiers distinct
14	<i>Only now</i> : compute the pre-authentication encoding, select each signature by a pinned key’s identifier, and verify both
15–16	Each pinned peer’s manifest reference is present and fresh, and the peer is active at the pinned epoch height
17–19	Resolve the receipt by invocation identifier in the receiver’s own store; its signature verifies, its tool name and argument hash agree, its kernel key is the pinned second-party key, and subject name and digest match the resolved body
20–22	Both verdicts resolve and agree; the lease resolves with matching issuer, expiry and scope digest and is unexpired; the tool name is in the action-class table; a receipt-backed class resolves its governance record
23–26	Binding cross-checks (Table 3), consistency model and anchor, co-signing mode; return the verified material

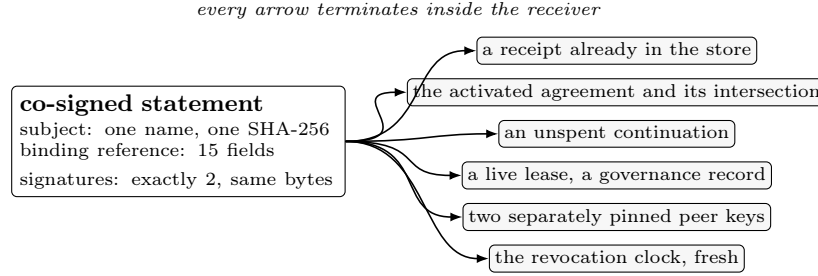


Figure 2: The mechanism. The statement carries no authority of its own: its subject, the binding fields the receiver compares, and each of its two keys resolve only against records the receiving kernel placed in its own stores before the request arrived.

them agreement-scoped, and maps every envelope-layer failure to **chio_treaty_unverified_required_evidence**. Table 3 and Section 8’s denial experiment report the hook, which decides live calls.

Signatures are verified last and the key lookup first, the opposite of the usual advice: neither decider can verify until it knows which keys to verify under, neither takes them from the envelope, and no gate before step 14 consults a secret, so the ordering costs nothing in soundness. The verifier reads those keys from its pin set and the hook from the activated agreement, so a rotation carried into one and not the other is admitted by whichever decider still holds the key the envelope names. Steps 17 to 19 are the verifier’s alone: neither the hook nor the kernel’s treaty verification resolves a receipt by invocation identifier or compares the subject digest, so on a live call the receipt binding rests on **local_receipt_sha256** and **remote_receipt_sha256** against the lineage bundle and the invocation record.

5.4 Consume, bind, co-sign

A replayed statement is validly signed, so no signature defeats replay. A single-use continuation does, and the receiver mints it before the call: a record binding the parent receipt and session anchor, the capability, the action class, the target tool, a nonce, the two kernel identities, and a validity window, whose digest is one of the 15 bound fields. It carries no signature and needs none: what authenticates it is residency in the receiver’s own store and the digest the request presents. Two operations run on one table keyed by that continuation identifier. *Consume* inserts, ignoring a conflict, and reads the affected row count; zero rows means the continuation was already spent and the call is denied with **chio_treaty_continuation_replay**. *Release* deletes the row for that identifier and that admission, and runs only on a later pre-dispatch denial, so a rejected call does not burn a continuation. Once dispatch commits the row stays: at-most-once, on purpose.

The receiver then builds its own admission report and binds the accepted material into the receipt it

signs, checking the statement’s `admission_report_sha256` only for well-formed hex before overwriting it with its *own* report’s digest.

The outbound statement is minted here, a different object: its subject is the receipt this call just produced, whose pre-authentication bytes the receiver authors and the peer signs without parsing. The local receipt is persisted first, so a refusal or transport failure is recorded against an existing receipt. The reference kernel mints it on every federated request that produced verified treaty material and fails the response when it cannot; a request denied before that mints none and returns its deny without failing. No admission path resolves it. Its second signature constrains nothing the receiver decided, since the Section 2 adversary holds the peer’s key: evidence, not prevention.

6 Security

Four properties carry the protocol, each over the adversary of Section 2 and reduced to an assumption of Table 1. All four concern the inbound statement; the outbound one gates the response, never the dispatch.

Admission binding. *If a receiving kernel dispatches a federated tool call, then both keys the activated agreement pins signed the presented statement S ’s pre-authentication bytes; every digest the request presented equalled that of the receiver-stored artifact its identifier resolved to, and S ’s binding reference names that digest for the agreement, the intersection, the continuation and any lineage bundle; every other compared leaf of S equals what the receiver computed for this request, save two bound to a receiver-held set, the lease expiry to the instants strictly after the receiver’s clock and the issuer to the agreement’s participants; and the continuation S names was unconsumed at admission.* Suppose not. Each equality is byte equality between a digest the receiver computed and one the statement carries, so a difference that passed means two objects share a SHA-256 image or a canonical encoding, contradicting ASSUME-SHA256 or the encoder’s injectivity under ASSUME-CANONICAL-JSON; a differing signature contradicts ASSUME-ED25519, a twice-consumed continuation ASSUME-SQLITE-ATOMICITY. Two residuals remain: completeness follows from exhaustiveness over the leaf type, and that the shipped hook realizes the enumerated predicate is a corpus on one fixture, fixing each leaf’s kind of comparison and not the quantity it names.

Nothing in the agreement puts the statement in that antecedent: the resolved co-signing mode adds the invocation record to required evidence only at exactly `bilateral_required`, nothing at `bilateral_if_cross_org`, and never the statement, which only an explicit required-evidence entry names. The kernel closes it above the hook, refusing a federated call the hook allowed carrying no verified statement. One passport key signs three preimages: the receipt signing body, the co-signing body and the pre-authentication encoding, so ASSUME-ED25519 carries these properties only with the three disjoint, which here is structural: a reserved first byte for the encoding, disjoint key sets for the others.

Receiver locality. *Every root, pin, agreement, action-class table and registry used to decide a request is state the receiver held before the request arrived; the request selects among them and contributes none.* Extraction reads a fixed set of key names and ignores the rest; each is an identifier, the argument of a total lookup over receiver-owned state.

Single use. *For a continuation identifier c , at most one admission ever reaches dispatch with c bound to it.* Under ASSUME-SQLITE-ATOMICITY exactly one of a set of concurrent inserts on a primary key creates the row; every other denies. Release deletes only the row keyed by c and the admission that took it, and runs only before dispatch (Section 4). And c is resident in the receiver’s store before the request, authenticated by residency and digest rather than signature, so the receiver decides how many authorizations exist. The reference path mints it there; origin minting, also permitted, owes an unpredictability premise.

Audience binding. *A statement accepted by receiver R is not accepted by a different receiver R' unless R' activated an identical agreement, resolved the same intersection, and holds the same continuation unconsumed.* The receiver’s identity enters through the continuation, not the statement’s name for it: the continuation R' resolved names R' as target and both kernels as participants of the agreement R' activated, which has exactly two, so R' admits only as its second, under the key it pins. Another validly co-signed statement from the same pair stops there too, naming a continuation the receiver does not know or has consumed.

Substitution over the binding reference. Table 3 is an executed corpus over the 15 binding fields, evidence PS-T12; every denial in it leaves the continuation unconsumed and hands no verified material to dispatch.

Table 3: Substituting one binding field at a time with a well-formed different value, both signatures valid over the substituted statement, as decided by the pre-dispatch hook. The signer list appears twice, as a set and in order, and the four conditional rows are compared only when the action class’s required-evidence set makes the admission resolve the artifact they bind. Each case resigns with both participant keys, so both signatures verify over the bytes the hook decides on; of the 22 cases one substitutes nothing and admits, so every denial is attributable to the field that changed. `binding_mismatch` and `unverified_evidence` abbreviate `chio_treaty_dsse_binding_mismatch` and `chio_treaty_unverified_required_evidence`.

Field	Receiver-side value it must equal	First code	Compared when
<code>treaty_id</code>	the agreement the receiver resolved for this request	<code>binding_mismatch</code>	always
<code>treaty_scope_sha256</code>	the digest the receiver computes over its own agreement record	<code>binding_mismatch</code>	always
<code>ladder_intersection_sha256</code>	the intersection digest the receiver resolved	<code>binding_mismatch</code>	always
<code>admission_report_sha256</code>	<i>nothing</i> : 64 hex characters, then overwritten by the receiver’s own report digest	none: the substituted call is admitted and dispatches	never
<code>continuation_sha256</code>	the resolved continuation’s digest	<code>binding_mismatch</code>	always
<code>action_class_id</code>	the action class the receiver resolved for this request	<code>binding_mismatch</code>	always
<code>consistency_model</code>	the model the resolved intersection fixed	<code>binding_mismatch</code>	always
<code>request_sha256</code>	this dispatch’s argument hash, and the predicate’s own <code>tool_args_hash</code>	<code>binding_mismatch</code>	always
<code>lease_refs</code>	the lease in the receiver’s own admission bundle	<code>binding_mismatch</code>	always
<code>governance_refs</code>	the governance record in that bundle	<code>binding_mismatch</code>	always
<code>signer_kernel_ids</code> , as a set	exactly two, equal as a set to the agreement’s participants	<code>binding_mismatch</code>	always
<code>signer_kernel_ids</code> , in order	the invocation record’s order; a reversal resolves the two pinned keys transposed, and strict envelope verification refuses that pairing before the ordered comparison runs	<code>unverified_evidence</code>	always
<code>lineage_bundle_sha256</code>	the lineage bundle the receiver resolved	<code>binding_mismatch</code>	lineage required
<code>outcome_sha256</code>	the outcome digest in the receiver’s invocation record	<code>binding_mismatch</code>	record required
<code>local_receipt_sha256</code>	the lineage bundle’s root receipt digest, and the invocation record’s	<code>binding_mismatch</code>	either required
<code>remote_receipt_sha256</code>	the lineage bundle’s leaf receipt digest, and the invocation record’s	<code>binding_mismatch</code>	either required

That table is one sub-object of the comparison set, not the set. A strict statement’s wire type has 55 leaves; the profile refuses 2, leaving 53 a statement may carry and 49 this call carries: 23 compared for equality against a named receiver-held value, 2 for membership in a receiver-held set, 13 against the statement alone, by a declared constant or a second leaf, and 11 against nothing the receiver holds. 245 executed cases fix that classification leaf by leaf.

Those 11 include the report digest, the consistency anchor, the policy and governance identifiers, and the subject digest, which only the conforming verifier of Section 5 checks. Each is still held to its declared shape: a substitution within it moves no decision, while a non-hexadecimal report digest and an empty policy identifier are denied. It is additive too: the 4 optional leaves this call omits are never read, so an adversary holding both keys adds them and the receiver admits.

A statement’s presentation window is the intersection of three intervals the receiver itself resolved: the agreement’s, the ladder intersection’s, and the continuation’s, each revocable and re-scoped per counterparty between calls, and after the first dispatch it is zero. The lease bounds it only from below, by the clock comparison above: the hook resolves no lease record and no governance record, so nothing the receiver stored bounds it above; the lease registry is the conforming verifier’s.

Revocation. Revocation is the only part of this protocol that depends on arrival. A kernel installs an origin’s signed snapshot only if its epoch strictly exceeds the installed one, by compare-and-swap, so neither a replay nor a lost concurrent install is possible, then applies its own freshness window to the issue time. Safety holds in every partition because both halves are local predicates that never wait for a message: no call is allowed against an observed revocation, and one evaluated against a stale snapshot is denied.

Liveness holds only under ASSUME-GOSSIP-FAIRNESS-PARTITION-BOUND, and bounds the time before a revocation is observed, not the number of evaluations, which is unbounded. With F the freshness

window, T the control-poll period, L one admitted call’s latency, and k the calls made before one is decided against the newer snapshot, a still-connected receiver denies within $T + kL$ of publication and a cut receiver within $F + T + kL$, its snapshot first ageing past F . At $F = 500$ ms, $T = 250$ ms and the median L and k of Section 8, those are 663.6 ms connected and 1163.6 ms cut.

7 What is mechanized

The decision core is a pure function: verify a capability, resolve grants, evaluate, sign. It is modeled in Lean 4 [8], whose theorem inventory carries 196 declarations and one explicit axiom, a collision-resistance idealization for ASSUME-SHA256. No serializer, protocol step, or kernel behavior is axiomatized; an unproved property is an assumption-registry entry.

Two results bear on this paper. Occurrence theorems make the predicate language fail closed: an atom unsupported, or undefined for the admission view, anywhere in a predicate forces denial on every view, by induction. The second is the comparison set of Section 6: the classification of every leaf of the co-signed predicate is total by exhaustiveness over the leaf type, and acceptance holds exactly when the statement gate and those comparisons do.

The classification is cross-checked, not transcribed: the executed corpus derives its enumeration from the wire type and fails unless the Lean module agrees leaf for leaf on the wire path and the comparison kind. What each leaf is compared against is prose on both sides, checked on neither, so a classification naming the wrong quantity would pass.

Lean does not verify the running Rust. These theorems are about the modelled predicate and classification; that the shipped hook realizes them is a corpus over one fixture, not a theorem. Four tripwires fail loudly when the two drift apart: named Rust functions mirrored to Lean definitions by content hash; 20 production functions extracted through a Rust-to-Lean pipeline with equivalence theorems that must rebuild byte-identically; 47 bounded model-checking harnesses over public entry points; and the cross-language encoder corpus of Section 8.

We report survivors, not percentages. Of 45 Lean mutants 33 were killed and 12 survived, each with an open issue; of 166 in the Rust proof-facing sources 160 were killed, 5 were unviable, and 1 survived. That campaign predates the classification’s entry in the mutation allowlist, so it bears on the occurrence theorems and not the classification, and the largest group of its survivors sits on the predicate language they are about.

8 Evaluation

Local measurements ran on one machine: 12 Neoverse-N1 cores, 46.9 GiB, Linux 6.17.0-1020-oracle aarch64, compiler 1.94.1 at the pinned toolchain. Percentiles interpolate between order statistics; each interval below is a percentile bootstrap interval for its median.

Two kernels in separate processes. Two kernels ran as separate operating-system processes, each with its own keys, stores, agreements, and receipt log, with a third process driving calls over iroh QUIC, relays disabled; the topology record reports a host count of 1. The co-signing and revocation hops are the shipped lanes unchanged; the request hop uses a transport defined for this experiment, since no shipped lane carries a tool call. Admitted calls: 100 calls, p50 206.803 ms, interval 205.924 to 209.998 ms, p99 234.520 ms at the driver, each costing 3 co-signing connections, counted by the receiver. The round trip is the receiver’s decision plus the co-signing hops it makes back to the origin after dispatching; its evaluation window, 203.787 ms at the median, contains those hops. Minting the per-call evidence costs the receiver 3.297 ms more, outside it.

Denied calls: 7 scenarios over the wire, 140 calls, p50 58.821 ms, p99 88.462 ms. They return `chio_treaty_missing_scope`, `chio_treaty_scope_hash_mismatch`, `chio_treaty_missing_intersection`, `chio_treaty_intersection_mismatch`, `chio_treaty_missing_required_evidence`, `request_smuggled_trust_root`, and `request_smuggled_dynamic_trust`. In every one the receiver’s dispatch counter stayed zero. 88 calls were retried and excluded, denied because the receiver’s snapshot had aged past its 500 ms freshness window against an origin publishing every 250 ms.

Revoking at the origin and measuring to the receiver’s first denial took 322.899 ms at the median over 10 repeats, interval 178.977 to 368.288 ms, a median of 2 calls. With the link cut the receiver serves until its snapshot leaves the freshness window and then denies, 744.681 ms at the median over 10 repeats, interval 609.423 to 916.110 ms, a median of 2 calls, counting a denial only once it persists for three further calls. On 3 of 4 revoke records and 0 of 10 cut records the denial also reports an unconfirmed pre-dispatch cleanup: the

Table 4: Local cost decomposition. The first two rows are complete kernel paths measured per invocation with both kernels in one process; the rest are components.

Path or component	p50	p99 or max	samples
Admitted call, both kernels in one process	13.863 ms	26.176 ms	200
Refused before dispatch	7.073 ms	18.646 ms	400
Receipt append to the store	4681.643 us	13566.644 us	500
Receipt signing	229.340 us	270.205 us	100
Receipt verification	337.470 us	344.582 us	100
Inbound two-signature statement verification	339.127 us	350.904 us	100
Cross-boundary admission decision	20.688 us	21.207 us	100

Table 5: A receiver composed from 4 parts an operator already has, driven under two wirings against the threat table of this paper, case by case where the composition has an analogue. The parts are a tool-call server, a peer identity pinned out of band, a policy engine that evaluates a local rule set and signs its decision, and a receiver-side replay table. *Composed* wires each part the way it documents itself; *hardened* adds checks no part requires and nothing in the composition records, among them a denylist over the context names the receiver owns and a replay table keyed by the identifier inside the signed bytes rather than one the caller picks. It is the wiring we wrote, not a bound on the wiring an operator could write. Admissions are over the 22 attacks the corpus drives, as 20 distinct experiments. Chio’s own record is at most 15.5 times heavier per call, a bound rather than a measurement, since the two stores are read by two procedures, and its side is read from the run above rather than re-measured here.

Property of Section 6	Composed	Hardened
Admission binding	fails	fails
Receiver locality	fails	holds
Single use	holds, weakened	holds
Audience binding	holds	holds
Attacks admitted	9	2
Durable bytes per call	763.9	866.3

budget unwind rather than the continuation release Section 4 orders, reporting a budget hold it cannot find, an invariant violation we do not explain. No call dispatched.

What admission costs locally. The shape of Table 4 is the result: the cryptography is not the expense. Appending the receipt durably costs 13.8 times what verifying the inbound statement costs, and 226 times the receiver’s decision over already resolved evidence, where the comparisons of Table 3 run. Durability dominates, which is the price of Section 4’s ordering. The refusal row is a lower bound: its fixture refuses on a unanimous deny verdict, an early exit reaching neither those comparisons nor the signature checks. Driven back to back for 60 s with one call outstanding, the same path admitted 3543 calls with 0 denials and 40.1 MiB of receipt-store growth: one call’s cost repeated, not a capacity.

The negative corpus is 20 in-process cases, each asserting a refused decision and its expected code: 8 through the dispatch-capable hook, 12 against the verifier directly, alongside 20 bilateral and 50 generic replay cases.

What a composition of existing parts reaches. Table 5 prices what an operator adds. Hardening closes 7 attacks and carries 3 of the four properties. Attacks divide by what closes them: a check over fields the request carries, or a field it does not have. Only the second is about the format: 10 of the 16 substitution rows have no carrier at all, one of them the report digest Chio never compares either. The residual witness, an unmodified admissible call against a receiver whose record for that agreement has advanced a version, is closed by a version field; the leaf enumeration of Section 7 counts the rest, so the carrier is settled by the decision, not added to it. Hardening is not free: the last row prices the second durable write it makes.

Verification by someone who was not there. The 50.6 KiB evidence package for the refund workflow verifies every receipt, statement, lease, and governance record in it with no kernel access, in 49.7 ms at the median over 100 runs. A frozen corpus of 14 receipt cases holds four independent implementations to the

same canonical body bytes and verdict.

Reproduction. Each run writes per-host commit and environment records, raw samples, and the macro file this paper compiles from, so a printed number and a retained one cannot drift. The comparison writes a duration only from a run on a clean input tree and an unloaded host, and otherwise records that it did not: its corpora and verdicts are above, its timings are not.

9 Limitations and related work

The cross-organization run of Section 8 put the two kernels in separate processes on one host, not on two separately administered machines. That removes the shared address space, store, and key material, which are what the claim is about; the shared operating system and clock source remain, and removing them changes the latencies. The rule does not depend on that experiment; the strength of the demonstration does.

Section 8’s alternative is one construction of those parts on one corpus, and its request schema is ours, so what it measures is what an operator must add, not a bound on what the parts reach. Continuation consumption is at-most-once, so a crash between consume and dispatch loses the call rather than repeating it: right for a refund, an avoidable retry for an idempotent read. The chain closes in the kernel for one hop: the next continuation in a longer chain is minted outside it. Denial paths below the verifier still return a rendered detail alongside the code; separating them is unfinished. Receipts batch into Merkle checkpoints in the RFC 6962 shape [14] with offline inclusion proofs, and that is the whole claim: local audit evidence with verifiable inclusion, not a public append-only log, with no anti-equivocation, so a kernel signing two contradictory checkpoints is detectable only by someone holding both.

Related work. Capability discipline without ambient authority is the tradition this protocol sits in [17, 23], saying exactly where a proof stops is seL4’s discipline [13], and attenuable credentials are old and good: trust management [9, 5], macaroons [3], and the modern token families [22, 6], which carry caveats without a kernel, a decision record, or a price. Kerberos [20] crosses the boundary by transferring authority, workload identity by minting a local credential for a foreign workload [19], and token exchange by converting one issuer’s token into another’s [11]: all three move authority, which is the step this protocol declines to take. Interchain light clients [10] solve pinned keys plus a monotone clock for chains, not tool calls, and policy engines decide from a request they are handed [7]. The envelope shapes we reuse [18, 21], transparent registration [4], and certificate transparency [14] answer how an artifact was built, who may register a statement, and how to publish verifiable records. None of them defines the object that is at once the authorization decision, the audit record, the accounting entry, and the evidence another organization’s kernel verifies against state it alone holds.

What we believe. The rule is smaller than the system that implements it, and we think it is the part that lasts. The receiver decides from facts only it holds; which names must cross follows from that decision rather than preceding it, and the record has to outlive the connection and be checkable by someone who trusts neither side.

References

- [1] Agent2Agent Project. A2A: Agent2agent protocol specification. Specification, 2025.
- [2] Anthropic. Model context protocol specification. Specification, 2024.
- [3] Arnar Birgisson et al. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014.
- [4] Henk Birkholz et al. An architecture for trustworthy and transparent digital supply chains. IETF SCITT working group, Internet-Draft, 2025.
- [5] Matt Blaze et al. The KeyNote trust-management system version 2. RFC 2704, 1999.
- [6] Geoffroy Couprie and Clément Delafargue. Biscuit: A decentralized authorization token. Specification, v3, 2023.
- [7] Joseph W. Cutler et al. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2024.
- [8] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *International Conference on Automated Deduction (CADE)*, 2021.
- [9] Carl M. Ellison et al. SPKI certificate theory. RFC 2693, 1999.
- [10] Christopher Goes. The interblockchain communication protocol: An overview. arXiv:2006.15918, 2020.
- [11] Michael B. Jones et al. OAuth 2.0 token exchange. RFC 8693, 2020.
- [12] Simon Josefsson and Ilari Liusvaara. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, IETF, 2017.
- [13] Gerwin Klein et al. seL4: Formal verification of an OS kernel. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2009.
- [14] Ben Laurie et al. Certificate transparency. RFC 6962, IETF, 2013.
- [15] Anders Rundgren et al. JSON canonicalization scheme (JCS). RFC 8785, IETF, 2020.
- [16] Jerome H. Saltzer et al. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4):277–288, 1984.
- [17] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

- [18] Secure Systems Lab. DSSE: Dead simple signing envelope. Secure Systems Lab specification, v1.0, 2021.
- [19] SPIFFE Project. SPIFFE: Secure production identity framework for everyone. Specification, federation and trust bundles, 2024.
- [20] Jennifer G. Steiner et al. Kerberos: An authentication service for open network systems. In *USENIX Winter Conference*, 1988.
- [21] Santiago Torres-Arias et al. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium*, 2019.
- [22] UCAN Working Group. UCAN: User controlled authorization networks. Specification, v1.0, 2025.
- [23] Robert N. M. Watson et al. Capsicum: Practical capabilities for UNIX. In *USENIX Security Symposium*, 2010.